

# Threads

A **thread** is the smallest unit of execution in a program. It allows multiple tasks to run concurrently in a program. Java provides built-in support for multithreading through the Thread class and the Runnable interface.

| Process                                                   | Thread                                                                                                   |
|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Independent execution unit with its own memory space      | Lightweight execution unit within a process                                                              |
| Heavier and slower as each process has separate resources | Threads share the same memory and resources of the parent process                                        |
| Example: Running a browser or a word processor.           | Example: A browser process may have separate threads for loading pages, rendering, and user interaction. |

**Thread Lifecycle:** A thread goes through the following states:

1. **New:** The thread is created using Thread or Runnable but has not yet started. Created using Thread t = new Thread();.
2. **Runnable:** After calling start(), the thread enters the runnable state. It is waiting for the CPU to schedule it.
3. **Running:** The thread is actively running on the CPU. This state depends on thread scheduling.
4. **Blocked:** The thread is waiting to acquire a resource or a lock or waiting for another thread to complete.
5. **Terminated:** The thread finishes execution or is explicitly stopped.

## Creating Threads (Two Main Ways)

**1. Extending the Thread Class:** A thread can be created by extending the Thread class and overriding its run() method.

**When to Use:** Use when the thread logic is tightly coupled with the class behavior. **Code Example:**

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running.");
    }
}

public class ThreadExample {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start(); // Starts the thread
    }
}
```

**2. Implementing the Runnable Interface:** A thread can also be created by implementing the Runnable interface. This approach decouples thread behavior from the class definition.

**When to Use:** Use when the class needs to inherit from another class, as Java does not support multiple inheritance. **Code Example:**

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Runnable thread is running.");
    }
}

public class RunnableExample {
    public static void main(String[] args) {
        Thread t = new Thread(new MyRunnable());
        t.start(); // Starts the thread
    }
}
```

## Thread Methods

1. **start():** Starts a thread by invoking its run() method.

```
Thread t = new Thread(new MyRunnable());
t.start();
```

2. **run()**: Contains the code that defines the task to be executed by the thread. If run() is called directly (without start()), it behaves like a normal method, not a thread.
3. **join()**: Causes the current thread to wait until another thread completes its execution.

```
t1.join(); // Main thread waits for t1 to finish
```

4. **sleep(ms)**: Puts the thread to sleep for the specified time in milliseconds.

```
Thread.sleep(1000); // Pauses the thread for 1 second
```

5. **yield()**: Hints the thread scheduler to allow other threads of equal priority to run.
6. **isAlive()**: Checks if a thread is still running.

```
Thread.yield();
```

```
if (t.isAlive()) {  
    System.out.println("Thread is running.");  
}
```

## Thread Synchronization

### Need for Synchronization:

- Synchronization is required to prevent **race conditions**, which occur when multiple threads access shared resources simultaneously, leading to unpredictable results.
- Synchronization ensures **mutual exclusion**, where only one thread accesses a critical section at a time.

### Key Concepts:

**1. Synchronized Block:** Locks only the critical section of code, making it more efficient than synchronizing the entire method.

- **Syntax:**

```
java Copy code  
  
synchronized(object) {  
    // Critical section  
}
```

- **Example:**

```
java Copy code  
  
class Shared {  
    void print() {  
        synchronized(this) {  
            System.out.println("Synchronized block");  
        }  
    }  
}
```

**2. Synchronized Method:** Locks the entire method so only one thread can execute it at a time.

- **Syntax:**

```
java Copy code  
  
synchronized void methodName() {  
    // Critical section  
}
```

- **Example:**

```
java Copy code  
  
class Shared {  
    synchronized void print() {  
        System.out.println("Synchronized method");  
    }  
}
```

**3. Static Synchronization:** Synchronizes static methods. Since static methods belong to the class, the **class-level lock** is used.

```
synchronized static void methodName() {  
    // Critical section  
}
```

**4. Locks:** Java's `java.util.concurrent.locks.Lock` interface provides more advanced thread synchronization mechanisms than synchronized blocks or methods. **Key Features:**

- Allows multiple lock conditions.
- Explicit lock and unlock control.

```
import java.util.concurrent.locks.ReentrantLock;  
  
class Shared {  
    private ReentrantLock lock = new ReentrantLock();  
  
    void print() {  
        lock.lock();  
        try {  
            System.out.println("Locked section");  
        } finally {  
            lock.unlock();  
        }  
    }  
}
```

## Inter-Thread Communication

Java provides the following methods to allow threads to communicate and coordinate their execution:

1. **wait():** Causes the current thread to **release the lock** and wait until another thread invokes `notify()` or `notifyAll()` on the same object. Must be called inside a synchronized block or method.

```
synchronized(object) {  
    object.wait();  
}
```

2. **notify()**: Wakes up **one waiting thread** (if any) that is waiting on the same object. The thread must acquire the lock to proceed.
3. **notifyAll()**: Wakes up **all threads** waiting on the same object. Only one thread proceeds after acquiring the lock.

```
synchronized(object) {
    object.notify();
}
```

```
synchronized(object) {
    object.notifyAll();
}
`##### **Example**:
```

## Thread Priority

### 1. Set and Get Priority

- i) **Setting Priority**: Use `setPriority(int priority)` to assign a priority to a thread. Java provides three constants:
  - `MIN_PRIORITY = 1`
  - `NORM_PRIORITY = 5` (default)
  - `MAX_PRIORITY = 10`
- ii) **Getting Priority**: Use `getPriority()` to retrieve the priority of a thread.

```
Thread t = new Thread();
t.setPriority(Thread.MAX_PRIORITY);
```

```
int priority = t.getPriority();
System.out.println("Thread Priority: " + priority);
```

### How Priorities Work:

1. Higher priority threads are more likely to be scheduled for execution, but priorities are platform-dependent.
2. Priority does not guarantee execution order, as it depends on the thread scheduler.

## Multithreading Utilities

**Concurrency Utilities (java.util.concurrent)**: Java provides a rich set of utilities to simplify thread management and synchronization:

### 1. Executor Framework: A framework for managing thread pools efficiently.

- **Benefits**:
  - Eliminates manual thread creation.
  - Optimizes resource utilization.
- **Key Interfaces**:
  - `Executor`: Submits tasks for execution.
  - `ExecutorService`: Manages and controls thread lifecycle.

```
ExecutorService executor = Executors.newFixedThreadPool(3);
executor.submit(() -> System.out.println("Task 1 executed"));
executor.shutdown();
```

### 2. Callable and Future:

- **Callable**: Similar to `Runnable`, but can return a result and throw exceptions.
- **Future**: Represents the result of an asynchronous computation.

```
Callable<Integer> task = () -> 42;
ExecutorService executor = Executors.newSingleThreadExecutor();
Future<Integer> result = executor.submit(task);
System.out.println(result.get()); // Retrieves the result
executor.shutdown();
```

### 3. Advanced Synchronization Utilities:

- **CountDownLatch:** Allows a thread to wait until a set of tasks complete. Example: Waiting for multiple services to initialize before proceeding.

```
CountDownLatch latch = new CountDownLatch(3);
Runnable task = () -> {
    System.out.println("Task executed");
    latch.countDown();
};
executor.submit(task);
latch.await(); // Wait until latch reaches 0
```

- **CyclicBarrier:** Synchronizes multiple threads by forcing them to wait at a barrier until all threads arrive. Example: Useful for parallel tasks that must finish a phase together.

```
CyclicBarrier barrier = new CyclicBarrier(3, () -> System.out.println("Barrier reached"));
```

- **Semaphore:** Controls access to a shared resource using permits. Example: Limiting access to a critical section.

```
Semaphore semaphore = new Semaphore(2); // 2 permits
semaphore.acquire(); // Acquires a permit
semaphore.release(); // Releases a permit
```

**4. ForkJoin Framework:** Designed for parallelism and dividing large tasks into smaller subtasks. **ForkJoinPool** manages threads efficiently for divide-and-conquer algorithms.

```
ForkJoinPool pool = new ForkJoinPool();
pool.invoke(new RecursiveTask<Integer>() {
    protected Integer compute() {
        return 1 + 2;
    }
});
```

## Deadlock and thread safe code

A deadlock occurs when two or more threads are waiting indefinitely for each other to release resources, preventing further execution. **Example:** Thread A locks Resource 1 and waits for Resource 2, while Thread B locks Resource 2 and waits for Resource 1. **How to Avoid Deadlocks:**

1. **Acquire Locks in a Consistent Order:** Always acquire locks in the same sequence in all threads.
2. **Use Timeout:** Use tryLock() with a timeout (from ReentrantLock) to prevent indefinite waiting.
3. **Avoid Nested Locks:** Minimize using locks inside other locks to reduce complexity.
4. **Deadlock Detection Tools:** Use monitoring tools or thread dumps to identify and resolve deadlocks.

**Thread-Safe Code:** ensures that shared data is accessed by threads in a way that prevents data corruption and unpredictable behavior. **Ways to Ensure Thread Safety:**

### 1. Synchronization:

Protect shared resources using synchronized blocks or methods.

```
java Copy code

synchronized void method() {
    // Thread-safe code
}
```

### 2. Atomic Classes:

Use classes like `AtomicInteger` for atomic operations without explicit synchronization.

```
java Copy code

AtomicInteger count = new AtomicInteger(0);
count.incrementAndGet();
```

### 3. Immutable Objects:

Design classes as immutable to ensure thread safety. Immutable objects cannot be modified once created.

```
java Copy code

final class ImmutableExample {
    private final int value;

    public ImmutableExample(int value) {
        this.value = value;
    }

    public int getValue() {
        return value;
    }
}
```

## Daemon Threads

Daemon threads are **background threads** that run in the background to perform tasks like garbage collection or resource monitoring. **Key Feature:** Daemon threads automatically terminate when all user threads finish execution. **Setting Daemon Threads:** Use `setDaemon(true)` before starting the thread.

```
Thread daemonThread = new Thread(() -> {
    while (true) {
        System.out.println("Daemon thread running...");
    }
});
daemonThread.setDaemon(true); // Set as daemon
daemonThread.start();
```

### Use Cases:

- Garbage Collection
- Monitoring Services
- Background Logging